

CODE IMPLEMENTATION

```
import { BrowserRouter, Route, Routes } from 'react-router-dom';

import { SurveillanceProvider } from '../context/SurveillanceContext';
import { Layout } from '../components/Layout';
import { Overview } from '../pages/Overview';
import { GenomeIndexing } from '../pages/GenomeIndexing';
import { Relatedness } from '../pages/Relatedness';
import { Mutations } from '../pages/Mutations';
import { TransmissionNetwork } from '../pages/TransmissionNetwork';
import { Clusters } from '../pages/Clusters';
import { SpreadSimulator } from '../pages/SpreadSimulator';
import { BudgetOptimizer } from '../pages/BudgetOptimizer';
import { Alerts } from '../pages/Alerts';
import { Samples } from '../pages/Samples';

export function App() {
  return (
    <SurveillanceProvider>
      <BrowserRouter>
        <Routes>
          <Route element={<Layout />}>
            <Route path="/" element={<Overview />} />
            <Route path="/genome-indexing" element={<GenomeIndexing />} />
            <Route path="/relatedness" element={<Relatedness />} />
            <Route path="/mutations" element={<Mutations />} />
            <Route path="/network" element={<TransmissionNetwork />} />
            <Route path="/clusters" element={<Clusters />} />
            <Route path="/spread-simulator" element={<SpreadSimulator />} />
            <Route path="/budget-optimizer" element={<BudgetOptimizer />} />
            <Route path="/alerts" element={<Alerts />} />
            <Route path="/samples" element={<Samples />} />
            <Route path="*" element={<Overview />} />
          </Route>
        </Routes>
      </BrowserRouter>
    </SurveillanceProvider>
  );
}
```

(alias) function GenomeIndexing(): React.JSX.Element {
 + import GenomeIndexing

```

import com.pathogen.model.Cluster;
import com.pathogen.model.GraphEdge;
import com.pathogen.model.GraphNode;

public class ClusterDetector {
    public static final class Result {
        public final Cluster[] clusters;
        public final SimpleHashTable<String> assignment;
        public Result(Cluster[] clusters, SimpleHashTable<String> assignment){this.clusters=clusters;}
    }

    public Result detect(GraphNode[] nodes, GraphEdge[] edges){
        IntList[] adj=new IntList[nodes.length];
        for(int i=0;i<nodes.length;i++)adj[i]=new IntList();
        for(int e=0;e<edges.length;e++){
            int a=index(nodes,edges[e].getSource()),b=index(nodes,edges[e].getTarget());
            if(a>=0&&b>=0){adj[a].add(b);adj[b].add(a);}
        }
        boolean[] visited=new boolean[nodes.length];
        String[][] groups=new String[nodes.length][];
        int gcount=0;
        for(int s=0;s<nodes.length;s++){
            if(visited[s])continue;
            int[] q=new int[nodes.length];int head=0,tail=0;q[tail++]=s;visited[s]=true;
            StringList members=new StringList();
            while(head<tail){
                int u=q[head++];members.add(nodes[u].getId());
                for(int z=0;z<adj[u].size();z++){int v=adj[u].get(z);if(!visited[v]){visited[v]=true;
                }}
            groups[gcount++]=members.toArray();
        }
        // Sort groups by descending size.
        for(int i=1;i<gcount;i++){
            String[] x=groups[i];int j=i-1;

```

```

import com.pathogen.model.GraphEdge;
import com.pathogen.model.GraphNode;
import com.pathogen.model.MstResult;

public class KruskalMST {
    public MstResult run(GraphNode[] nodes, GraphEdge[] edges){
        GraphEdge[] sorted=copy(edges);
        // Insertion sort is deliberately explicit for the DSA engine.
        for(int i=1;i<sorted.length;i++){
            GraphEdge x=sorted[i];int j=i-1;
            while(j>=0 && sorted[j].getWeight()>x.getWeight()){sorted[j+1]=sorted[j];j--;}
            sorted[j+1]=x;
        }
        UnionFind uf=new UnionFind(nodes.length);
        EdgeList selected=new EdgeList();double total=0;int considered=0;
        for(GraphEdge e:sorted){
            considered++;
            int a=index(nodes,e.getSource()),b=index(nodes,e.getTarget());
            if(uf.union(a,b)){selected.add(e);total+=e.getWeight();if(selected.size()==nodes.length)
            }
        }
        // Build component member arrays.
        String[][] components=new String[nodes.length][];
        int count=0;
        for(int i=0;i<nodes.length;i++){
            int root=uf.find(i);int pos=-1;
            for(int c=0;c<count;c++){if(components[c]!=null && components[c].length>0){
                int idx=index(nodes,components[c][0]); if(uf.find(idx)==root){pos=c;break;}
            }}
            if(pos<0){components[count++]=new String[]{nodes[i].getId()};}
            else components[pos]=append(components[pos],nodes[i].getId());
        }
        // Remove nulls and sort by size.
        String[][] clean=new String[count][];
        for(int i=0;i<count;i++)clean[i]=components[i];
        for(int i=1;i<clean.length;i++){String[] x=clean[i];int j=i-1;while(j>=0&&clean[j].length<x

```

```

J RelatednessCalculator.java X
src > main > java > com > pathogen > algorithm > J RelatednessCalculator.java > {} com.pathogen.algorithm
1 package com.pathogen.algorithm;
2
3 import com.pathogen.model.GenomeSample;
4 import com.pathogen.model.RelatednessResult;
5
6 /**
7  * CO / Module-2 + Module-3:
8  * - k-mer Jaccard / MinHash-style scoring (string algorithms)
9  * - Needleman-Wunsch global alignment (advanced DP)
10 */
11 public class RelatednessCalculator {
12     private final KmerIndexer indexer = new KmerIndexer();
13
14     public RelatednessResult build(GenomeSample[] samples, String method, int k) {
15         int n=samples.length;
16         double[][] matrix=new double[n][n];
17         String[] ids=new String[n];
18         for(int i=0;i<n;i++){ ids[i]=samples[i].getId(); matrix[i][i]=1.0; }
19         double min=1,max=0,sum=0; int pairs=0;
20         for(int i=0;i<n;i++){
21             for(int j=i+1;j<n;j++){
22                 double score=scorePair(samples[i].getGenome(),samples[j].getGenome(),method,k);
23                 matrix[i][j]=score; matrix[j][i]=score;
24                 if(score<min)min=score;if(score>max)max=score;sum+=score;pairs++;
25             }
26         }
27         return new RelatednessResult(ids,matrix,method,k,pairs,pairs==0?0:min,pairs==0?0:max,pairs==0?0:sum/pairs);
28     }
29
30     public double scorePair(String a,String b,String method,int k){
31         if("alignment".equalsIgnoreCase(method)) return needlemanWunschIdentity(a,b,1,-1,-2);
32         if("minhash".equalsIgnoreCase(method)) return minHashStyle(a,b,k);
33         return kmerJaccard(a,b,k);
34     }
35
36     public double kmerJaccard(String a,String b,int k){
37         SimpleHashTable<Boolean> A=indexer.uniqueKmers(a,k), B=indexer.uniqueKmers(b,k);
38         SimpleHashTable.EntryIterator<Boolean> it=A.iterator();
39         int inter=0;
40         while(it.hasNext()){

```

```

public class SimpleHashTable<V> {
    private static final int INITIAL_CAPACITY = 1024;

    private static final class Entry<V> {
        String key;
        V value;
        Entry<V> next;
        Entry(String key, V value, Entry<V> next) {
            this.key = key;
            this.value = value;
            this.next = next;
        }
    }
}

```

```

public class TarjanBridgeFinder {
    private GraphNode[] nodes;
    private GraphEdge[] edges;
    private IntList[] adj;
    private int[] disc, low, parentEdge;
    private boolean[] visited;
    private int timer;
    private Edgelist bridges;

    public Bridge[] find(GraphNode[] nodes, GraphEdge[] edges, SimpleHashTable<String> assignment) {
        this.nodes = nodes; this.edges = edges;
        adj = new IntList[nodes.length]; for (int i = 0; i < nodes.length; i++) adj[i] = new IntList();
        for (int e = 0; e < edges.length; e++) { int a = index(edges[e].getSource()), b = index(edges[e].getTarget());
        disc = new int[nodes.length]; low = new int[nodes.length]; parentEdge = new int[nodes.length];
        visited = new boolean[nodes.length]; for (int i = 0; i < nodes.length; i++) { disc[i] = -1; parentEdge[i] = -1; }
        timer = 0; bridges = new Edgelist();
        for (int i = 0; i < nodes.length; i++) if (!visited[i]) dfs(i);
        Bridge[] out = new Bridge[bridges.size()];
        for (int i = 0; i < bridges.size(); i++) {
            GraphEdge e = bridges.get(i);
            Bridge b = new Bridge(String.format("BR-%02d", i+1), e);
            b.setClusterSource(valueOrDash(assignment, e.getSource())); b.setClusterTarget(valueOrDash(assignment, e.getTarget()));
            int[] sizes = sideSizes(e);
            b.setSideASize(sizes[0]); b.setSideBSize(sizes[1]); b.setAffectedSamples(sizes[0] + sizes[1]);
            out[i] = b;
        }
    }
}

```